

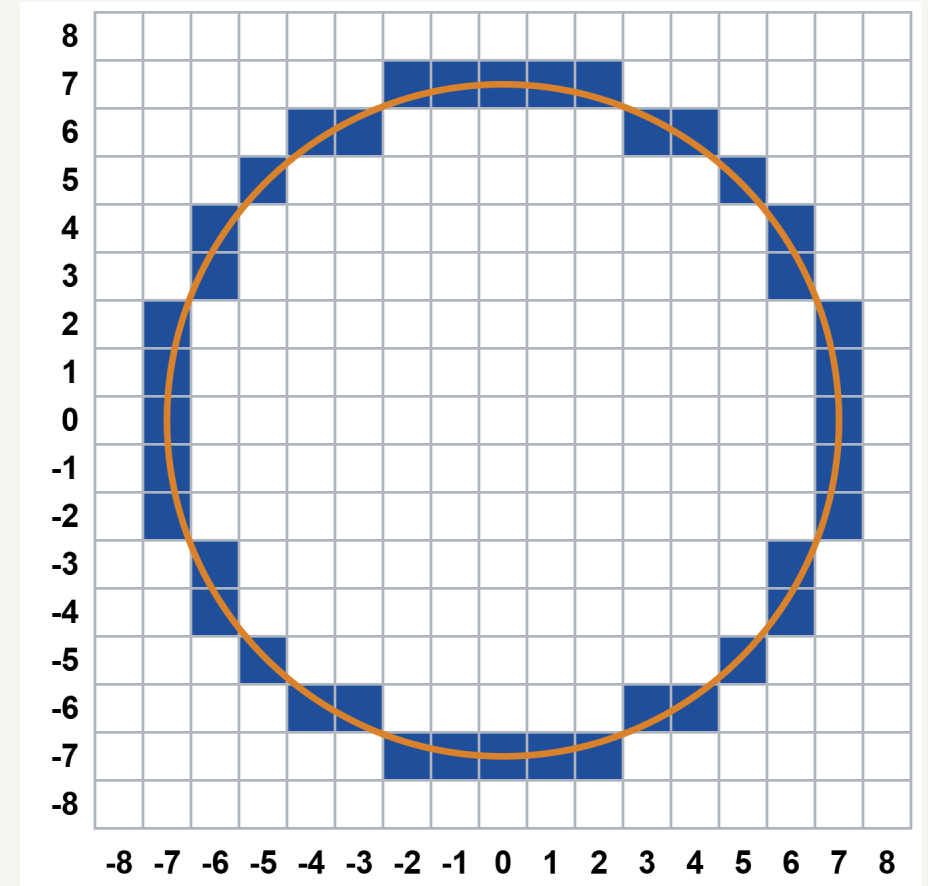
0613-403 • Week 3 • Lecture 5

Midpoint Line and Midpoint Circle

Ekramul Haque Amin

Lecturer, AI and Data Science

Dhaka International University



Warm-up: Bresenham in 3 minutes

Q1

Line (0, 0) to (6, 4). Find dx, dy and d_1 .

Q2

$d_1 \geq 0$. Which pixel comes next: S or T? What is d_2 ?

Q3

Why does Bresenham need no rounding?

Today we see the same idea from a new angle (the midpoint) and use it to draw circles.

Today's plan

Part 1

Midpoint line

- The midpoint test
- Line function $F(x, y)$
- Proof of the formulas
- Same as Bresenham

Part 2

Comparison

- Direct vs DDA vs Bresenham
- Same line, three methods

Part 3

Midpoint circle

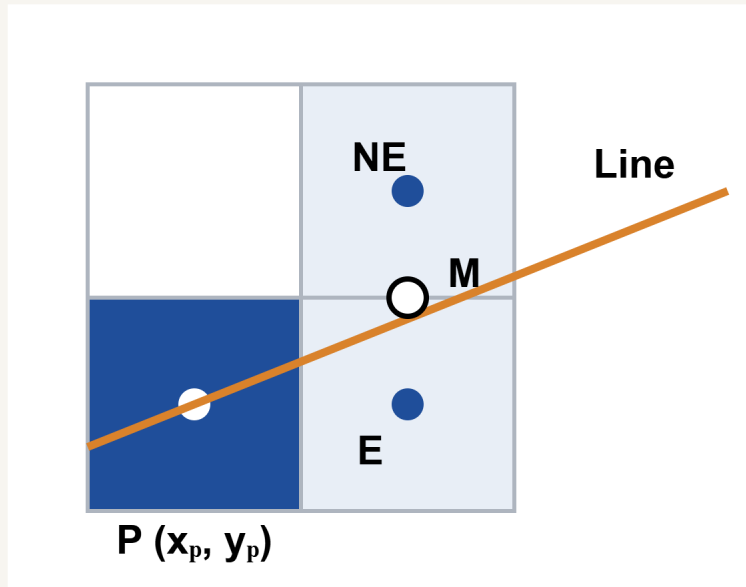
- 8-way symmetry
- Circle function
- Proof of p_0 and updates
- Example, exercise, code

Part 1

The midpoint line algorithm

The same pixels, explained with one test point

The idea: test the midpoint



From pixel P the next pixel is E (east) or NE (north-east).

$M = (x_p + 1, y_p + \frac{1}{2})$ is the point halfway between them.

Line passes above M

NE is closer $\rightarrow$ choose NE

Line passes below M

E is closer $\rightarrow$ choose E

Midpoint variables and their meaning

Symbol	Meaning
(x_p, y_p)	The pixel just plotted
$E = (x_p + 1, y_p)$	East candidate: right
$NE = (x_p + 1, y_p + 1)$	North-east candidate: right and up
$M = (x_p + 1, y_p + \frac{1}{2})$	Midpoint between E and NE
$F(x, y) = dy \cdot x - dx \cdot y + dx \cdot b$	Line function: 0 on the line
$d = F(M)$	Decision variable: value of F at the midpoint
$\Delta E, \Delta NE$	Amounts added to d after choosing E or NE

Midpoint proof 1: the line function

Start from $y = (dy/dx) \cdot x + b$

Multiply by dx : $dx \cdot y = dy \cdot x + dx \cdot b$

Move everything to one side:

$$F(x, y) = dy \cdot x - dx \cdot y + dx \cdot b$$

$F = 0$ for every point on the line.

Sign of F

A point below the line has a smaller y , so $-dx \cdot y$ is larger: $F > 0$.

A point above the line: $F < 0$.

Midpoint proof 2: the decision at M

Put $M = (x_p + 1, y_p + \frac{1}{2})$ into $F(x, y) = dy \cdot x - dx \cdot y + dx \cdot b$:

$$d = dy(x_p + 1) - dx(y_p + \frac{1}{2}) + dx \cdot b \dots (1)$$

M is below the line $\rightarrow F(M) > 0$ (proof 1)

M is above the line $\rightarrow F(M) < 0$

$d > 0 \rightarrow$ choose NE; $d \leq 0 \rightarrow$ choose E

Why?

$d > 0$: M is below the line, so the line passes closer to NE (the upper pixel).

$d < 0$: M is above the line, so the line passes closer to E.

$d = 0$: the line goes through M; we pick E.

Midpoint proof 3: update after choosing E

$E = (x_p + 1, y_p)$. The next midpoint is $(x_p + 2, y_p + \frac{1}{2})$:

$$d_{\text{new}} = dy(x_p + 2) - dx(y_p + \frac{1}{2}) + dx \cdot b$$

Subtract (1):

$$d_{\text{new}} - d = dy(x_p + 2 - x_p - 1) - dx(y_p + \frac{1}{2} - y_p - \frac{1}{2})$$

$$= dy(1) - dx(0)$$

$$d_{\text{new}} = d + dy, \text{ so } \Delta E = dy$$

Notice

x moved by 1, y did not move.

The $dx \cdot b$ terms cancel, so b is never needed.

Only one addition per step.

Midpoint proof 4: update after choosing NE

NE = $(x_p + 1, y_p + 1)$. The next midpoint is $(x_p + 2, y_p + 3/2)$:

$$d_{\text{new}} = dy(x_p + 2) - dx(y_p + 3/2) + dx \cdot b$$

Subtract (1):

$$d_{\text{new}} - d = dy(x_p + 2 - x_p - 1) - dx(y_p + 3/2 - y_p - 1/2)$$

$$= dy(1) - dx(1)$$

$$d_{\text{new}} = d + dy - dx, \text{ so } \Delta NE = dy - dx$$

Notice

Both x and y moved by 1.

$dy - dx$ is negative for $0 \leq m < 1$, so d goes down after NE.

These are half of Bresenham's $2dy$ and $2(dy - dx)$.

Midpoint proof 5: start value and scaling

First midpoint: $(x_1 + 1, y_1 + \frac{1}{2})$

$$d_1 = dy(x_1 + 1) - dx(y_1 + \frac{1}{2}) + dx \cdot b$$

$$= [dy \cdot x_1 - dx \cdot y_1 + dx \cdot b] + dy - dx/2$$

$$= F(x_1, y_1) + dy - dx/2$$

$F(x_1, y_1) = 0$ (it is on the line), so $d_1 = dy - dx/2$

Fraction! Multiply d , ΔE and ΔNE by 2:

$$d_1 = 2dy - dx, \Delta E = 2dy, \Delta NE = 2(dy - dx)$$

Surprise

These are exactly Bresenham's numbers.

Multiplying by 2 does not change the sign, so every decision stays the same.

Midpoint and Bresenham: the same algorithm

Midpoint on (1, 1) to (8, 5): $d_1 = 1$, $\Delta E = 8$, $\Delta NE = -6$

x	y	d	Decision	New d
1	1	1	$d > 0 \rightarrow NE$	-5
2	2	-5	$d \leq 0 \rightarrow E$	3
3	2	3	$d > 0 \rightarrow NE$	-3
4	3	-3	$d \leq 0 \rightarrow E$	5
5	3	5	$d > 0 \rightarrow NE$	-1
6	4	-1	$d \leq 0 \rightarrow E$	7
7	4	7	$d > 0 \rightarrow NE$	1
8	5	—	Stop	—

Only difference

When $d = 0$ exactly, Bresenham picks T (up) and the midpoint method picks E. Both are correct: the line passes through the midpoint.

Same pixels as Bresenham
Example 1 in Lecture 4.

Part 2

Direct vs DDA vs Bresenham

Three ways to draw the same line

Comparing the three line methods

	Direct method	DDA	Bresenham / Midpoint
Main formula	$y = mx + b$	$y_{i+1} = y_i + m$	$d_{i+1} = d_i + 2dy$ or $d_i + 2(dy - dx)$
Work per pixel	Multiply, add, round	2 additions, 2 roundings	1 addition, 1 comparison
Number type	Floating point	Floating point	Integers only
Rounding	Every pixel	Every pixel	None
Steep lines	Gaps unless handled	No gaps	No gaps (swap x and y)
Speed	Slowest	Faster	Fastest
Long lines	Exact but slow	Error can build up	Exact
Used in	Teaching only	Simple software	Graphics hardware

Same line, three methods: (1, 1) to (8, 5)

x	Direct $y = (4x + 3)/7$	DDA y	Bresenham y	Pixel (all three)
1	1	1	1	(1, 1)
2	1.57	1.57	2	(2, 2)
3	2.14	2.14	2	(3, 2)
4	2.71	2.71	3	(4, 3)
5	3.29	3.29	3	(5, 3)
6	3.86	3.86	4	(6, 4)
7	4.43	4.43	4	(7, 4)
8	5	5	5	(8, 5)

All three give the same pixels. The difference is the arithmetic: decimals and rounding vs one integer test.

Part 3

The midpoint circle algorithm

Same midpoint idea, now for circles

The circle equation

Centre at the origin

$$x^2 + y^2 = r^2$$

Every point (x, y) on the circle is exactly r away from $(0, 0)$.

Centre at (x_c, y_c)

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

Trick: draw the circle at $(0, 0)$, then add x_c and y_c to every pixel.

So we only need to solve one problem: a circle of radius r centred at the origin.

Why not just use the equation?

1. Direct (polynomial) method

$$y = \sqrt{r^2 - x^2} \text{ for } x = 0, 1, 2, \dots, r$$

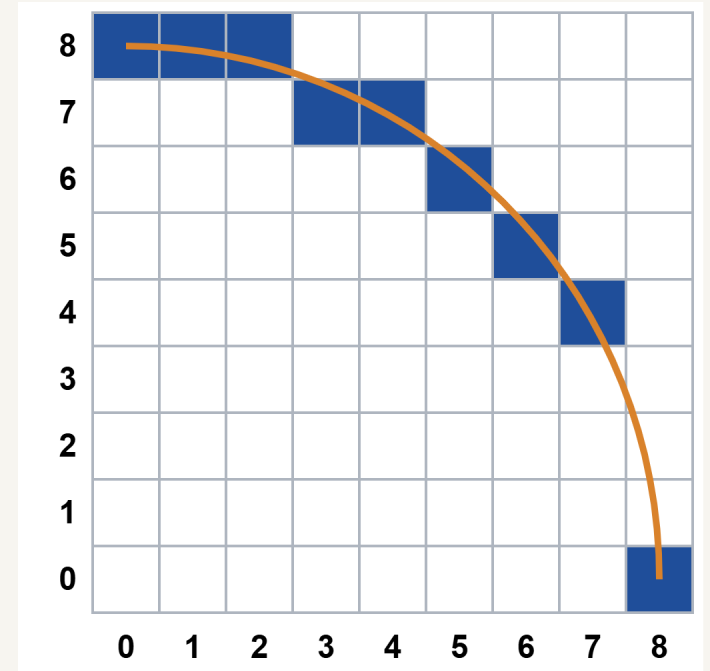
A square root at every pixel: slow.

2. Polar (trigonometric) method

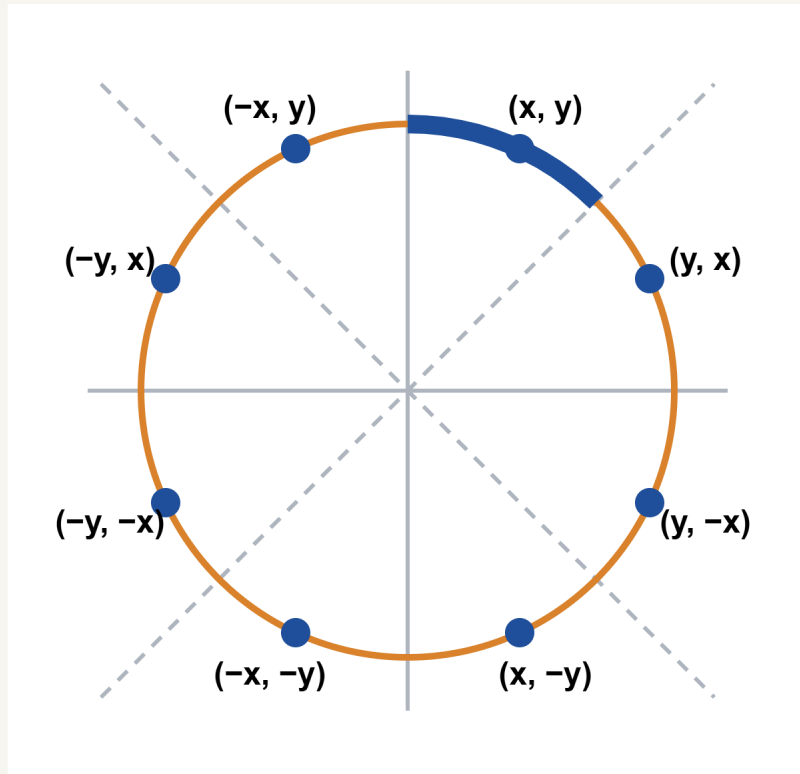
$$x = r \cos \theta, y = r \sin \theta$$

sin and cos at every pixel: even slower.

Both also leave gaps where the circle is steep. Look at $x = 7 \rightarrow 8$.



8-way symmetry



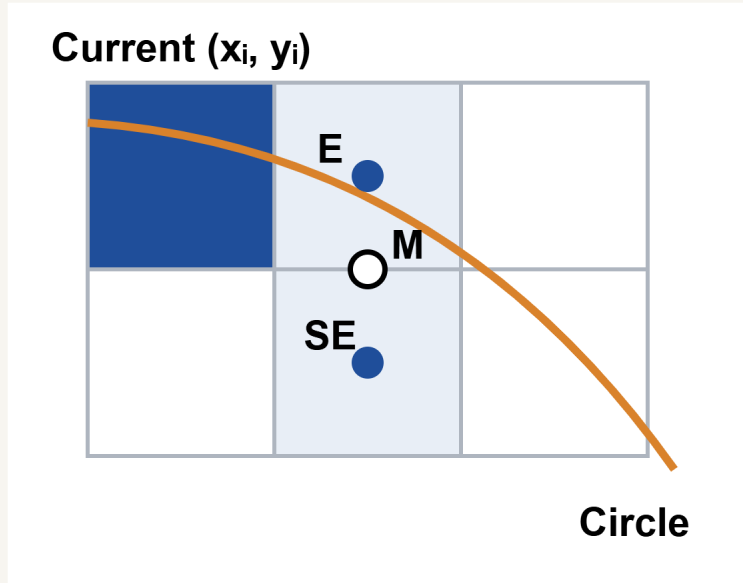
A circle looks the same when flipped about the x-axis, the y-axis and the lines $y = x$ and $y = -x$.

So one computed point (x, y) gives 8 pixels:

$(\pm x, \pm y)$ and $(\pm y, \pm x)$

We compute only the blue octant (from the top, 90° , down to 45°) and copy the rest.

Two choices in our octant



Start at the top $(0, r)$. In this octant x goes up by 1 every step, and y stays or goes down by 1.

$$E = (x_i + 1, y_i)$$

East: y stays the same

$$SE = (x_i + 1, y_i - 1)$$

South-east: y goes down by 1

Test the midpoint $M = (x_i + 1, y_i - \frac{1}{2})$ between them.

Every variable and its meaning

Symbol	Meaning
(x_c, y_c)	Centre of the circle
r	Radius (an integer)
(x_i, y_i)	Current pixel, measured from the centre
E, SE	The two candidate pixels for the next step
M	Midpoint between E and SE : $(x_i + 1, y_i - \frac{1}{2})$
$F(x, y)$	Circle function: $x^2 + y^2 - r^2$
p_i	Decision parameter: F evaluated at M

The circle function $F(x, y) = x^2 + y^2 - r^2$

$$F < 0$$

Point is inside the circle

$$F = 0$$

Point is on the circle

$$F > 0$$

Point is outside the circle

$p_i < 0 \rightarrow$ choose E

M is inside, so the circle passes above M. E is closer.

$p_i \geq 0 \rightarrow$ choose SE

M is outside (or on), so the circle passes below M. SE is closer.

Proof 1: the decision parameter

Put $M = (x_i + 1, y_i - \frac{1}{2})$ into F :

$$p_i = (x_i + 1)^2 + (y_i - \frac{1}{2})^2 - r^2$$

This is the value we test at every step.

Meaning

p_i tells us whether the midpoint is inside or outside the circle.

We never compute it like this in the loop: the next slides show how to update it with additions.

Proof 2: update when E is chosen

Next midpoint: $(x_i + 2, y_i - \frac{1}{2})$

$$p_{i+1} = (x_i + 2)^2 + (y_i - \frac{1}{2})^2 - r^2$$

$$p_{i+1} - p_i = (x_i + 2)^2 - (x_i + 1)^2$$

$$= (x_i^2 + 4x_i + 4) - (x_i^2 + 2x_i + 1)$$

$$p_{i+1} = p_i + 2x_i + 3$$

Why so short?

y did not change, so the y-terms cancel.

Only x moved from $x_i + 1$ to $x_i + 2$.

Proof 3: update when SE is chosen

Next midpoint: $(x_i + 2, y_i - 3/2)$

x-part (as before): $2x_i + 3$

y-part: $(y_i - 3/2)^2 - (y_i - 1/2)^2 = -2y_i + 2$

$p_{i+1} = p_i + 2x_i + 3 - 2y_i + 2$

$$p_{i+1} = p_i + 2(x_i - y_i) + 5$$

Check the y-part

$$\begin{aligned} & (y^2 - 3y + 9/4) - (y^2 - y + 1/4) \\ &= -2y + 2 \end{aligned}$$

Proof 4: the starting value p_0

First pixel: $(x_0, y_0) = (0, r)$. First midpoint: $(1, r - \frac{1}{2})$

$$p_0 = 1^2 + (r - \frac{1}{2})^2 - r^2$$

$$= 1 + r^2 - r + \frac{1}{4} - r^2$$

$$= \frac{5}{4} - r$$

$$p_0 = 1 - r \text{ (rounded)}$$

Why can we drop the $\frac{1}{4}$?

Every update adds a whole number, so p always keeps the same fraction $\frac{1}{4}$.

Testing $1\frac{1}{4} - r < 0$ gives the same answer as $1 - r < 0$ for integer r .

Formula card: midpoint circle

Start

$$p_0 = 1 - r$$

at (0, r)

If $p < 0$ (choose E)

$$p_{i+1} = p_i + 2x_i + 3$$

y stays

If $p \geq 0$ (choose SE)

$$p_{i+1} = p_i + 2(x_i - y_i) + 5$$

y goes down by 1

x_i and y_i are the values before the step. Stop when $x > y$. Then plot all 8 symmetric points.

Midpoint circle algorithm

1. Input centre (x_c, y_c) and radius r .
2. $x = 0, y = r, p = 1 - r$
3. Plot the 8 symmetric points of (x, y) , each shifted by (x_c, y_c) .
4. If $p < 0$: $p = p + 2x + 3$. Else: $p = p + 2(x - y) + 5$ and $y = y - 1$.
5. $x = x + 1$.
6. Repeat steps 3–5 while $x \leq y$.

Exam table

Columns: x, y, p , decision, new p .

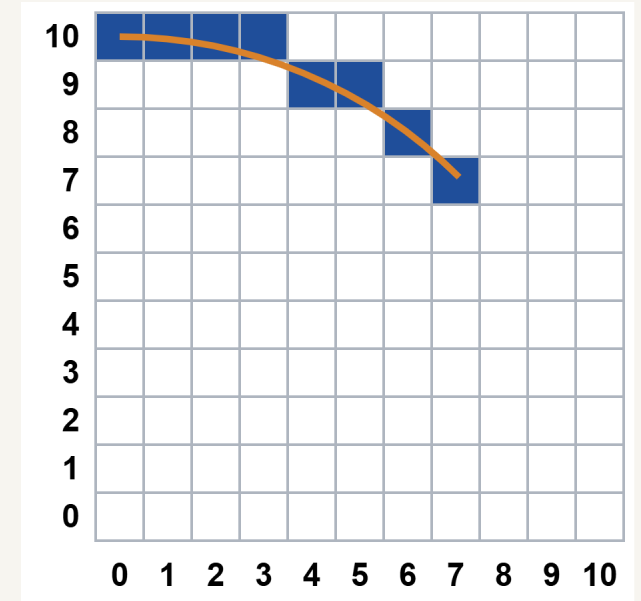
Stop when $x > y$ (past the 45° line).

Then list the 8 symmetric points.

Example: circle with $r = 10$, centre $(0, 0)$

$p_0 = 1 - 10 = -9$. Start at $(0, 10)$.

x	y	p	Decision	New p
0	10	-9	$p < 0 \rightarrow E, y \text{ stays}$	-6
1	10	-6	$p < 0 \rightarrow E, y \text{ stays}$	-1
2	10	-1	$p < 0 \rightarrow E, y \text{ stays}$	6
3	10	6	$p \geq 0 \rightarrow SE, y - 1$	-3
4	9	-3	$p < 0 \rightarrow E, y \text{ stays}$	8
5	9	8	$p \geq 0 \rightarrow SE, y - 1$	5
6	8	5	$p \geq 0 \rightarrow SE, y - 1$	6
7	7	—	Stop: $x = y$	—



Using symmetry: the whole circle

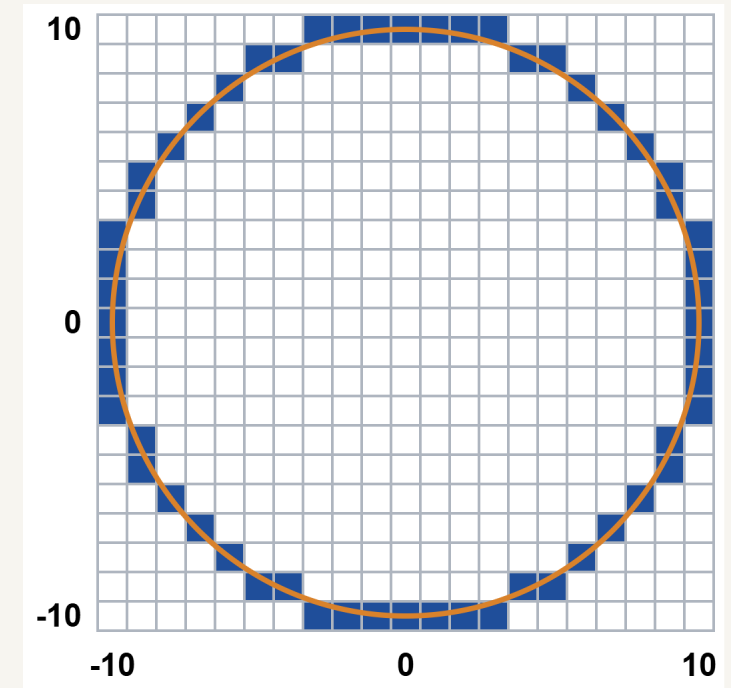
From the octant pixel (3, 10) we get 8 pixels:

(3, 10) (10, 3) (-3, 10) (-10, 3)

(3, -10) (10, -3) (-3, -10) (-10, -3)

For centre $(x_c, y_c) = (50, 40)$, add 50 to x and 40 to y: (3, 10) $\rightarrow$ (53, 50).

8 octant pixels gave the whole circle: 56 different pixels.



Two ways to write the update (Lab-04)

Lecture (use old x, y)

Compute p first, then move:

$$\text{E: } p = p + 2x + 3$$

$$\text{SE: } p = p + 2(x - y) + 5$$

then $x = x + 1$ (and $y = y - 1$)

Lab-04 code (use new x, y)

Move first, then compute p:

$$\text{E: } p = p + 2x + 1$$

$$\text{SE: } p = p + 2(x - y) + 1$$

x and y are already updated

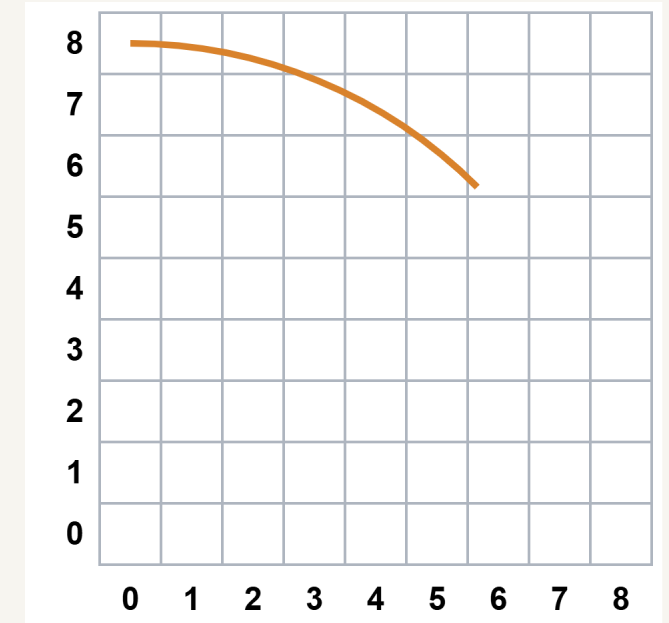
Same algorithm: new $x = \text{old } x + 1$ turns $2x + 3$ into $2x + 1$. Both draw the same pixels.

Class exercise: your turn

Use the midpoint circle algorithm for a circle with $r = 8$, centre $(0, 0)$.

1. Find p_0
2. Fill in the table until $x > y$
3. Shade the octant pixels
4. Write the 8 symmetric points of $(2, 8)$

Time: 6 minutes.



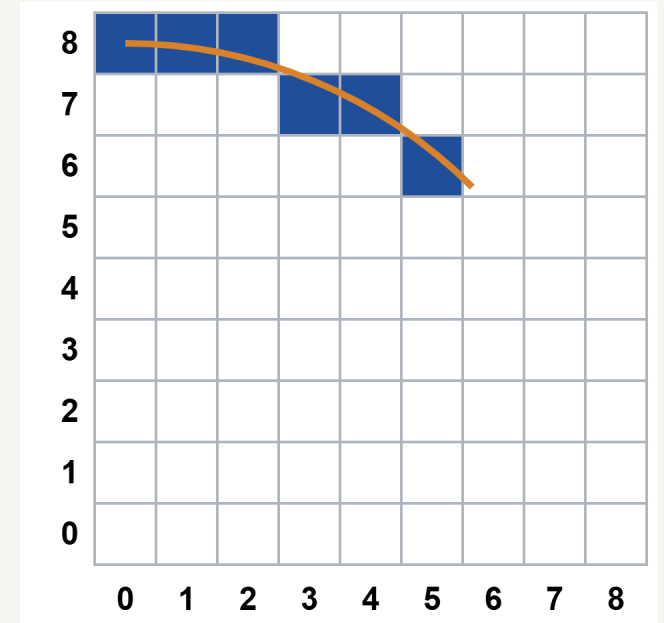
Exercise answer: $r = 8$

$p_0 = 1 - 8 = -7$. Start at $(0, 8)$.

x	y	p	Decision	New p
0	8	-7	$p < 0 \rightarrow E, y \text{ stays}$	-4
1	8	-4	$p < 0 \rightarrow E, y \text{ stays}$	1
2	8	1	$p \geq 0 \rightarrow SE, y - 1$	-6
3	7	-6	$p < 0 \rightarrow E, y \text{ stays}$	3
4	7	3	$p \geq 0 \rightarrow SE, y - 1$	2
5	6	2	$p \geq 0 \rightarrow SE, y - 1$	5

Next would be $(6, 5)$: $x > y$, so stop.

Symmetric points of $(2, 8)$: $(2, 8)$ $(8, 2)$ $(-2, 8)$ $(-8, 2)$ $(2, -8)$ $(8, -2)$ $(-2, -8)$ $(-8, -2)$



Midpoint circle in C++ with OpenGL

```
void plot8(int xc, int yc, int x, int y) {
    glVertex2i(xc + x, yc + y); glVertex2i(xc + y, yc + x);
    glVertex2i(xc - x, yc + y); glVertex2i(xc - y, yc + x);
    glVertex2i(xc + x, yc - y); glVertex2i(xc + y, yc - x);
    glVertex2i(xc - x, yc - y); glVertex2i(xc - y, yc - x);
}

void midpointCircle(int xc, int yc, int r) {
    int x = 0, y = r, p = 1 - r;
    glBegin(GL_POINTS);
    while (x ≤ y) {
        plot8(xc, yc, x, y);
        if (p < 0) p += 2 * x + 3;           // E
        else { p += 2 * (x - y) + 5; y--; } // SE
        x++;
    }
    glEnd();
}
```

Midpoint circle: advantages and disadvantages



Advantages

- **Integers only: no square root, no sin or cos**
- **One addition and one comparison per step**
- **8-way symmetry: only 1/8 of the work**
- **No gaps in the circle**



Disadvantages

- **Only for circles (ellipses need a new version)**
- **Radius and centre must be integers**
- **Pixels can repeat on the axes and 45° lines**

Key points from today

1. The midpoint line method tests $F(x, y)$ at the midpoint and gives the same formulas as Bresenham.
2. Direct $\rightarrow$ DDA $\rightarrow$ Bresenham: each is faster than the last; Bresenham uses integers only.
3. A circle has 8-way symmetry, so we compute only one octant (x from 0 to y).
4. $F(x, y) = x^2 + y^2 - r^2$: negative inside, zero on, positive outside.
5. $p_0 = 1 - r$; $p < 0 \rightarrow E$, add $2x + 3$; $p \geq 0 \rightarrow SE$, add $2(x - y) + 5$ and $y - 1$.
6. Draw at the origin, then add (xc, yc) to every pixel.

Practice questions

1. Use the midpoint line algorithm for the line from (2, 2) to (10, 7).
2. Compare the direct method, DDA and Bresenham in four points.
3. Use the midpoint circle algorithm to find the octant pixels for $r = 6$.
4. Use the midpoint circle algorithm for $r = 12$, centre (20, 15). List the first octant and the 8 points of the first pixel.
5. Derive $p_0 = 1 - r$ and the two update formulas.
6. Why do we compute only one octant? Write the 8 symmetric points of (x, y).

Coming up

Next lecture

Ellipses and area filling

Midpoint ellipse algorithm, then boundary fill, flood fill and scan-line fill

Lab

Lab 4: midpoint circle

Code the midpoint circle in Python with PyOpenGL and draw concentric circles

Questions?

Thank you

Ekramul Haque Amin · Lecturer, AI and Data Science